FLUTTER DEVELOPER PRACTICAL ASSESSMENT

Expected Completion Time	1 working day
Application	Country & Weather Explorer

Assessment Requirement

Build a Flutter application that consumes public/open APIs and demonstrates a well-structured, production-style approach. The application should be functional, visually polished, and easy to use.

1. Firebase Authentication

- Email and password login
- Form validation
- Login error handling
- Maintain the authenticated session
- Logout functionality

2. Countries API Integration

Use any suitable public countries API, such as REST Countries API.

- Fetch and display a list of countries
- Show country name, flag, capital, and region
- Open a Country Details screen when a country is selected

3. Country Details

- Country name and flag
- Capital and region
- Population
- Currency
- Latitude and longitude

The selected country latitude and longitude should be reused for the weather and map functionality.

4. Weather API Integration

Use OpenWeather or another suitable public weather API. Fetch current weather using the selected country's latitude and longitude.

- Temperature
- Weather condition
- Humidity
- Wind speed

The weather section should load independently. If the weather API fails, the Country Details screen should remain usable.

5. Map Integration

- Display the selected country/location on a map using latitude and longitude

- Show a marker for the selected location
- OpenStreetMap with flutter_map, or another suitable map solution, may be used

6. Search

- Allow users to search countries by name
- Handle matching results
- Handle empty search and no-result states

7. State Management

Use a proper state management solution such as Riverpod, Bloc/Cubit, Provider, or another suitable approach.

- Handle loading, success, empty, and error states
- Keep business logic separate from UI widgets
- Avoid managing the complete application using only setState()

8. API & Code Structure

- Keep API calls separate from UI widgets
- Use proper Dart models for API responses
- Separate networking, repository/data logic, state management, and presentation code
- Keep API URLs and keys in an appropriate configuration layer rather than repeatedly hardcoding them

9. Error & Loading Handling

- No internet connection
- Firebase authentication failure
- Countries API failure
- Weather API failure
- API timeout
- Empty or invalid API response
- Missing/null data without application crashes
- Retry option where appropriate
- Independent loading states for country and weather data

10. Local Storage

Implement at least one small local-storage feature using SharedPreferences, Hive, or another suitable solution:

- Last 5 searched countries
- Favorite countries
- Last selected country

11. UI / UX

The application should have a clean, modern, and professional UI/UX. It should look like a presentable mobile application, not a basic Flutter demo.

- Consistent spacing, typography, colors, and component styling
- Clean country cards/list items and details layout
- Clear actions and navigation
- Responsive layout
- User-friendly forms and search
- Appropriate icons
- Good empty, loading, and error states
- Reusable UI components where appropriate

- Consistent visual design across Login, Country List, Country Details, Weather, Map, and Search

Minimum Required Features

- Firebase login and logout
- Countries API integration
- Country list and details
- Country search
- Latitude/longitude handling
- Weather API integration
- Map integration
- State management
- Loading and error handling
- Basic local storage
- Clean and professional UI/UX

Bonus Task - Optional Backend API

This task is optional and should only be attempted after completing the main Flutter requirements. The candidate may create and integrate a backend API for any section of the application.

- Favorites API - save, retrieve, and delete favorite countries
- Recent Searches API - store and retrieve recent country searches
- User Preferences API - store preferences such as favorite country or temperature unit
- Custom Country API - expose selected country information through a custom backend API

Any suitable backend technology may be used, including Node.js/Express, Firebase Cloud Functions, NestJS, .NET, Python, or another appropriate framework.